AGENTIC AI BUILDERS FELLOWSHIP

STUDENT EXERCISE SHEET

Session 04: Data Structures Deep Dive

Phase 1 — Python for AI · Skill: Model real data with lists, dictionaries, and sets

1 — BEFORE YOU START

Open your Session 03 file (session_03_smart_unit_converter.py). Run it and confirm it works without errors. You should see the unit converter menu appear in the terminal and be able to make a conversion.

This exercise does not modify your Session 03 code. You will create a brand-new file called session_04_[your_domain].py. Everything from Session 03 stays intact.

Run these two checks before you begin:

```
# Check 1 — confirm your virtual environment is active
$ python --version
Python 3.10.x   (any 3.10 or higher is fine)

# Check 2 — confirm session 03 still works
$ python session_03_smart_unit_converter.py
=== Smart Unit Converter ===
...
```

No new packages to install. Everything in Session 04 is built into Python.

2 — KEY CONCEPTS IN PLAIN ENGLISH

These are the new ideas introduced in Session 04. Read them before you look at the steps.

list

An ordered container that holds multiple items. You access each item by its position number, starting from zero.

Example: names = ['Priya', 'Alex', 'Meera'] — names[0] gives 'Priya', names[1] gives 'Alex'.

index

The position number of an item in a list. Python always starts counting from zero, not one.

Example: names[0] is the first item. names[2] is the third. There is no names[3] in a three-item list.

dictionary

A container that stores pairs of names and values. You look up data by name, not by position number.

Example: contact = {'name': 'Priya', 'phone': '9876543210'} — contact['name'] gives 'Priya'.

key-value pair

One entry in a dictionary. The key is the name (e.g. 'phone'). The value is the data (e.g. '9876543210'). They are separated by a colon.

Example: 'phone': '9876543210' is one key-value pair. 'phone' is the key. '9876543210' is the value.

nested data structure

A data structure inside another data structure. A list of dictionaries is the most common pattern in production code — a list where every item is a dictionary.

Example: contacts = [{'name': 'Priya', ...}, {'name': 'Alex', ...}] — contacts[0]['name'] gives 'Priya'.

.get() method

A safe way to read a value from a dictionary. If the key you ask for does not exist, it returns a default value instead of crashing your program.

Example: contact.get('mobile', 'Not provided') — returns 'Not provided' if 'mobile' is missing. contact['mobile'] would crash.

set

A container that never holds duplicate values. Every time you add an item that already exists, the set ignores it silently.

Example: tags = {'college', 'engineering', 'college'} — the set stores only {'college', 'engineering'}.

named constant

A variable written in ALL_CAPS at the top of your file that holds a configuration value. You reference it by name instead of hardcoding the number in multiple places.

Example: MAX_RECORDS = 50 at the top of the file. If the limit changes, you edit one line — not twenty.

3 — WHAT YOU SAW IN THE DEMO

In the demo, we built a Contact Book CLI. Here is what we covered:

- We stored multiple contacts as a list of dictionaries — each contact had name, phone, email, and a nested list of tags.
- We placed MAX_CONTACTS = 100 at the top of the file before any functions — the Named Constants Pattern.
- We wrote add_contact() to check the limit against MAX_CONTACTS before appending a new dictionary to the list.
- We used contact.get('phone', 'Not provided') in list_all_contacts() to safely read fields that might be missing — never crashing on absent keys.
- We used sorted(contacts_list, key=lambda c: c['name']) to display contacts alphabetically without changing the original list.
- We built get_all_tags() using a set() — looping through every contact's tags and adding each one to the set. Duplicates were removed automatically.
- We used a list comprehension with an if condition in search_contact() to filter matching records without a manual loop.
- We used contacts_list.pop(i) inside delete_contact() to remove a record by its index after finding it with enumerate().

You will apply the same patterns to your own domain project.

4 — WHAT YOU WILL BUILD

You will build the Data Model layer for your Phase 1 domain project — the project idea you chose in Session 01.

This is not a new project. It is the data foundation that your Session 10 capstone CLI tool will stand on. You are building the storage and retrieval system that every future session will extend.

By the end of this exercise, your file will store a list of records as a list of dictionaries, support adding, searching, listing, and tag display, and follow the named constants and .get() safety patterns from the demo.

5 — STARTER SPEC

Input	User runs the program. The demo data is pre-loaded. No live user input required.
Output	Terminal output showing all records, a search result, a tag list, and an add confirmation.
Format	Terminal only. No file I/O in this session — that comes in Session 06.

Min records	At least 3 pre-loaded records in your list of dictionaries.
Min fields	At least 4 fields per record (e.g. name, description, rating, tags).

6 — STEP-BY-STEP BUILD GUIDE

Each step is one action. Complete each step, run your code, and confirm it works before moving to the next.

Step 1 — Create your new file

Create a new Python file called session_04_[your_domain].py. For example: session_04_recipes.py or session_04_movies.py. Open it in VS Code.

```
# Mac / Linux
$ touch session_04_recipes.py

# Windows (PowerShell)
> New-Item session_04_recipes.py
```

Step 2 — Add your docstring and named constant

At the very top of your file, write a docstring describing what the file does. Then write your named constant.

```
"""
Session 04 — Data Structures Deep Dive
Agentic AI Builders Fellowship · Phase 1: Python for AI

My domain: [describe your project domain here]
"""

# PRODUCTION PATTERN: Named constant — configuration lives at the top of
the file.
# Change this in one place if the limit ever changes.
MAX_RECORDS = 50
```

Replace [describe your project domain here] with your own domain — recipes, movies, books, games, etc.

MAX_RECORDS is written in ALL_CAPS. This is the naming convention for constants in Python.

Step 3 — Build your list of dictionaries

Below your named constant, create a variable called records. Assign a list of at least 3 dictionaries to it. Each dictionary must have at least 4 fields, and one field must be a list called 'tags'.

```
# This is our in-memory database: a list of dictionaries.
# Each dictionary is one record. The list holds all records.
records = [
    {
        'title':  'Butter Chicken',
        'cuisine': 'Indian',
        'rating':  5,
        'tags':    ['spicy', 'popular', 'chicken']
    },
```

```
    {
        'title':  'Pasta Carbonara',
        'cuisine': 'Italian',
        'rating':  4,
        'tags':    ['popular', 'quick', 'pasta']
    },
    {
        'title':  'Miso Soup',
        'cuisine': 'Japanese',
        'rating':  4,
        'tags':    ['quick', 'healthy', 'vegetarian']
    },
]
```

Replace these with records from your own domain. Keep the same structure — a list on the outside, a dictionary for each record, and a 'tags' field that holds a list.

Step 4 — Write add_record()

Write a function that checks the limit and adds a new dictionary to the list.

```
def add_record(records_list, title, **fields):
    """Add a new record dictionary to the records list."""
    if len(records_list) >= MAX_RECORDS:
        print(f'[LIMIT] List is full. Maximum: {MAX_RECORDS}.')
        return
    new_record = {'title': title, **fields}
    records_list.append(new_record)
    print(f'[ADDED] {title} saved.')
```

***fields lets you pass any extra keyword arguments. This makes your function flexible for different record structures.*

len(records_list) >= MAX_RECORDS is the guard clause — it checks the constraint before doing any work.

Step 5 — Write list_all_records()

Write a function that prints all records sorted alphabetically by their title field. Use .get() for every field you print.

```
def list_all_records(records_list):
    """Print all records sorted alphabetically by title."""
    if not records_list:
        print('[EMPTY] No records yet.')
        return
    sorted_records = sorted(records_list, key=lambda r: r['title'])
    print(f'\n=== All Records ({len(sorted_records)}) ===')
    for record in sorted_records:
        print(f"  {record['title']}")
        # Use .get() for every field — safe access even if a field is
missing
```

```
        for key, value in record.items():
            if key != 'title':
                print(f'    {key}: {value}')
```

sorted() returns a new sorted list. It does NOT change your original records list.

.get() is used here as a safety habit — once you build larger systems with data from APIs, some fields will sometimes be missing.

Step 6 — Write search_records()

Write a function that searches for records whose title contains the search term. Use a list comprehension with an if condition.

```
def search_records(records_list, search_term):
    """Return a list of records whose title contains the search term."""
    results = [
        record for record in records_list
        if search_term.lower() in record['title'].lower()
    ]
    return results
```

Read the list comprehension in plain English: 'give me every record where the search term appears in the title'. The .lower() on both sides makes the search case-insensitive.

Step 7 — Write display_tags()

Write a function that collects all tags from all records and returns them as a set. Duplicates disappear automatically.

```
def display_tags(records_list):
    """Return a set of every unique tag used across all records."""
    all_tags = set()   # set() with parentheses — NOT {} which makes an
empty dict
    for record in records_list:
        for tag in record.get('tags', []):
            all_tags.add(tag)
    return all_tags
```

set() with parentheses creates an empty set. {} alone creates an empty dictionary.

record.get('tags', []) returns an empty list if the tags field is missing — safe access, no crash.

Step 8 — Write main() and run the demo

Write a main() function that calls all four functions to show the full system working.

```
def main():
    """Run a demo of the full data model."""
    print(f'=== {__file__} — Session 04 Demo ===\n')

    # Show starting data
    list_all_records(records)
```

```
    # Add a new record
    print('\n--- Adding a new record ---')
    add_record(records, 'Pad Thai', cuisine='Thai', rating=4,
               tags=['noodles', 'popular', 'quick'])

    # Search
    print('\n--- Searching for pasta ---')
    results = search_records(records, 'pasta')
    for r in results:
        print(f"  Found: {r['title']}")

    # Show unique tags (set)
    print('\n--- All unique tags ---')
    unique_tags = display_tags(records)
    print(f'  Tags: {sorted(unique_tags)}')


if __name__ == '__main__':
    main()
```

if __name__ == '__main__': means: only run main() if you launched this file directly. If another file imports this file, main() does NOT run automatically.

Step 9 — Run your program

Save the file. Run it in the terminal. Verify the output matches what you expect.

```
$ python session_04_recipes.py
```

You should see: all records listed alphabetically, the add confirmation, a search result, and a sorted list of unique tags.

7 — EXPECTED OUTPUT

Your output will use your domain, not recipes. This example shows the shape and format your output should match.

```
=== session_04_recipes.py — Session 04 Demo ===

=== All Records (3) ===
  Butter Chicken
    cuisine: Indian
    rating: 5
    tags: ['spicy', 'popular', 'chicken']
  Miso Soup
    cuisine: Japanese
    rating: 4
```

```
      tags: ['quick', 'healthy', 'vegetarian']
    Pasta Carbonara
      cuisine: Italian
      rating: 4
      tags: ['popular', 'quick', 'pasta']

  --- Adding a new record ---
  [ADDED] Pad Thai saved.

  --- Searching for pasta ---
    Found: Pasta Carbonara

  --- All unique tags ---
    Tags: ['chicken', 'healthy', 'noodles', 'pasta', 'popular', 'quick',
  'spicy', 'vegetarian']
```

Notice: records appear alphabetically (Butter Chicken, Miso Soup, Pasta Carbonara — not the order you wrote them). Tags are unique — 'popular' and 'quick' appear once each even though multiple records had them.

8 — STRETCH GOAL (optional)

Stretch Goal 1: Add a delete_record() function.

Use enumerate() to loop through the records list with both the index and the record. When you find a title match, use records_list.pop(i) to remove it. Print '[DELETED] {title} removed.' If no match is found, print '[NOT FOUND]'.

Stretch Goal 2: Add a filter_by_tag() function.

Write a function that takes a tag name as a string and returns all records that contain that tag in their tags list. Use a list comprehension with a condition. Hint: 'quick' in record.get('tags', []) checks if a tag is present.

Stretch Goal 3: Add a top_rated() function.

Write a function that returns the record with the highest rating. Use max() with a key function: max(records_list, key=lambda r: r.get('rating', 0)).

9 — SUBMISSION RUBRIC

Criterion	Partial	Complete
List of dictionaries structure	Records are stored as plain variables or strings, not as a list	A variable called records holds a list. Each item in the list is a

Criterion	Partial	Complete
	of dictionaries. The nested structure is missing.	dictionary with at least 4 fields including 'tags'.
Named constant used correctly	MAX_RECORDS is missing, or it appears inside a function instead of at the top of the file.	MAX_RECORDS is defined in ALL_CAPS at the top of the file, above all functions. add_record() checks against it.
.get() used for dictionary reads	Dictionary values are read using square brackets everywhere (e.g. record['phone']). No .get() calls.	All dictionary reads that could fail use .get() with a default value. No KeyError risk in the output functions.
set used in display_tags()	display_tags() returns a list with duplicates, or is not written. No set used.	display_tags() creates a set(), loops through all tags, adds each one, and returns the set. Duplicates are absent from the output.
Program runs without errors	Running python session_04_[domain].py produces a traceback or incomplete output.	Running python session_04_[domain].py produces the full expected output: list, add confirmation, search result, and unique tags.

10 — SUBMISSION INSTRUCTIONS

- File name: session_04_[your_domain].py (e.g. session_04_recipes.py, session_04_movies.py)
- Submit to the community within 48 hours of the live session.
- Include: a screenshot of your terminal showing the full expected output from Step 9.
- Push to your GitHub repo and include the URL in your submission. Use a meaningful commit message: 'S04: add data model for [your domain]'

11 — GETTING UNSTUCK

The most common mistake in this exercise is creating an empty set with {} instead of set(). If display_tags() gives you an empty dictionary instead of tags, check that line.

Most Common Error 1: KeyError: 'title'

Your dictionary does not have a 'title' key, or you mis-spelled it (e.g. 'Title' with capital T).

Fix: Print one record to check its keys: print(records[0].keys()). Verify the field names match exactly what you typed when creating the dictionary.

Most Common Error 2: TypeError: unhashable type: 'list'

You tried to add a list directly to a set. Sets can only hold individual strings or numbers — not lists.

Fix: Check your display_tags() function. The inner loop should add individual tag strings: all_tags.add(tag). Not: all_tags.add(record['tags']).

Most Common Error 3: set() creates an empty dict (or no tags appear)

You wrote all_tags = {} instead of all_tags = set(). Empty curly braces make a dictionary, not a set.

Fix: Change {} to set(). These look similar but they are different objects.

Most Common Error 4: Search returns nothing

The search term comparison is case-sensitive, or you forgot to call .lower() on one side.

Fix: Confirm your search condition reads: search_term.lower() in record['title'].lower(). Both sides must be lowercase.

1. Re-read the demo code on GitHub: session_04_contact_book.py. Your code should follow the same structure.
2. Post your error message + the 5 lines before it in the community Discord. Include which step you were on.
3. Compare your terminal output to the Expected Output in Section 7. What is different? That difference tells you what is wrong.
4. Tag Prakash in the community — response within 24 hours.

12 — GLOSSARY

New terms introduced in Session 04. All future sessions will use these without re-defining them.

Term	Definition
list	An ordered container that holds multiple items. Items are accessed by position number (index), starting from zero.
index	The position number of an item in a list. The first item is always index 0.
dictionary	A container that stores key-value pairs. Items are accessed by name (key), not by position.
key-value pair	One entry in a dictionary. The key is the name; the value is the data. Separated by a colon.
nested data structure	A data structure inside another data structure. The most common pattern: a list of dictionaries.
.get()	A safe dictionary method. Returns a default value instead of crashing if the key does not exist.

set	A container that only holds unique values. Duplicates are automatically ignored when added.
named constant	A variable in ALL_CAPS at the top of a file that holds a configuration value. Never hardcoded inside functions.